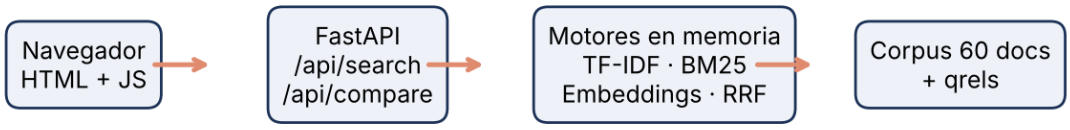
TAREA 4: INTEGRACIÓN DE MOTORES DE BÚSQUEDA

Sistema fullstack con TF-IDF, BM25, Embeddings y búsqueda híbrida (RRF): comparación de calidad, latencia p50/p95 y prueba con hasta 50 usuarios concurrentes

Francisco Corzo Pérez · Matrícula 233296

CORZO_PEREZ_FRANCISCO_233296_C2_T4_INTEGRACION



Resumen. Se construyó un buscador fullstack (FastAPI + frontend web) que sirve los motores de las tareas anteriores sobre un corpus de 60 documentos de 106 a 121 palabras. El híbrido RRF obtuvo la mejor calidad (nDCG@5 0.908), BM25 fue el mejor motor léxico y los embeddings ganaron en consultas semánticas pero fallaron en términos literales. Con 50 usuarios concurrentes y una pausa de 500 ms entre consultas, la latencia de extremo a extremo fue p50 4.4 ms y p95 12.2 ms, sin errores; sin pausa (estrés) subió a p50 97 ms y p95 442 ms por saturación de la máquina de pruebas, no de los motores. **Aviso de honestidad:** el corpus es sintético (redactado para el proyecto) y los qrels se derivan de reglas; no se hizo web scraping contra el sitio real porque el entorno no tuvo acceso, así que no se reclaman los puntos extra.

1. Corpus y consultas de evaluación

Corpus. 60 documentos tipo nota periodística sobre Chiapas, en 10 temas × 6 documentos (agua, café y cacao, turismo, salud, riesgos naturales, tecnología, infraestructura, medio ambiente, economía, cultura). Cada texto tiene entre 106 y 121 palabras (media 114.1; 6,844 en total), cumpliendo los mínimos del enunciado (≥50 documentos, ≥100 palabras). Los textos son originales, redactados para este proyecto: los lugares son reales (Cañón del Sumidero, Soconusco, Palenque…) pero los sucesos son ilustrativos y no deben leerse como noticias reales.

Consultas y juicios de relevancia. 20 consultas en tres tipos: 6 léxicas (expresión literal: «Cañón del Sumidero», «Soconusco», «Aedes aegypti», «marimba», «Lacandona», «pozol»), 8 semánticas (parafrasean sin repetir las palabras del documento: «temblor fuerte», «enfermedad transmitida por zancudos»…) y 6 mixtas («exportación de café orgánico», «cacao y chocolate»…). Relevancia graduada 1–3: en las léxicas es 3 si el documento contiene la expresión; en las demás se asigna por subtema. **Limitación:** como los qrels salen del mismo diseño del corpus, sirven para comparar motores entre sí, no como medida absoluta de calidad.

2. Desarrollo del sistema

Backend (FastAPI). Carga el corpus y construye los 4 índices al arrancar (≈3.8 s, ≈1 GB de memoria por el modelo de vectores). Endpoints: `/api/search?q&engine&k`, `/api/compare` (los 4 motores y el solapamiento Jaccard), `/api/documents[/id]`, `/api/engines`, `/api/health` y `/api/stats` (p50/p95/p99 acumulados por motor). Cada búsqueda mide su tiempo con `perf_counter` y lo registra en un recolector con ventana de 20,000 muestras seguro entre hilos.

Motores. TF-IDF (tf=f/|d|, idf=ln(N/df), coseno) y BM25 (k1=1.5, b=0.75) con índice invertido en NumPy y pesos precalculados; Embeddings con el promedio de los vectores de palabras de spaCy es_core_news_lg (300 d) y coseno sobre una matriz normalizada; Híbrido con RRF (k=60) sobre BM25 y Embeddings. Usan dos preprocesamientos distintos: el léxico (sin acentos, stopwords salvo 13 palabras como «no» o «sin», stemming Snowball) y el de palabras completas con acentos y sin stemming para los vectores.

Frontend. Página web sin frameworks con tres pestañas: Buscar (selector de motor, resaltado de términos, vista del documento), Comparar (los 4 motores lado a lado con su latencia y Jaccard) y Latencia (p50/p95/p99 del servidor). Las funciones puras (resaltado, formato, Jaccard) se prueban con Node y el flujo completo con jsdom contra la API real.

Pruebas. 36 pruebas de pytest (los motores contra fórmulas ingenuas independientes, el cálculo exacto de RRF, las métricas, la API en modos threadpool e inline, los requisitos del corpus y el scraper con HTML de muestra, además de una prueba con los vectores reales) y 7 pruebas unitarias más 1 de DOM en el frontend. Todas pasan.

3. Comportamiento de los motores

Motor	nDCG@5	MRR	R@5	nDCG léxicas	nDCG semánticas	nDCG mixtas
TF-IDF	0.860	0.967	0.719	1.000	0.779	0.829
BM25	0.868	0.967	0.729	1.000	0.797	0.830
Embeddings	0.762	0.838	0.697	0.649	0.856	0.749
Híbrido RRF	0.908	1.000	0.790	1.000	0.900	0.826

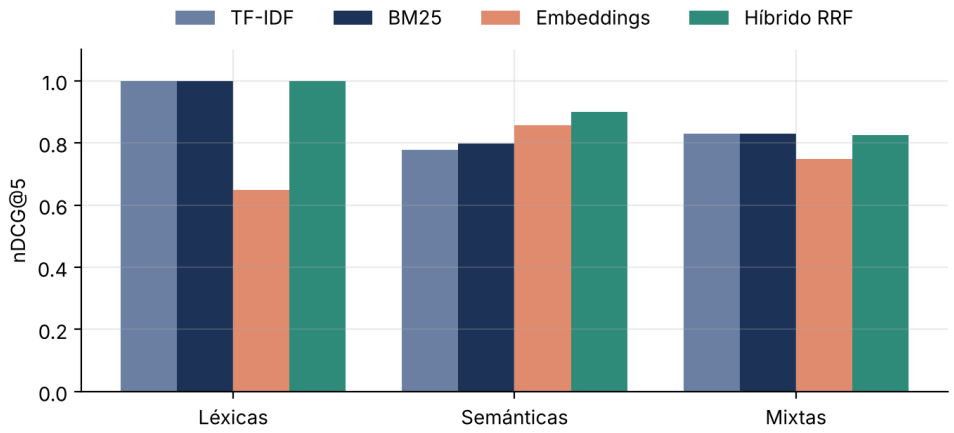


Figura 1. nDCG@5 por tipo de consulta (20 consultas).

TF-IDF y BM25 son perfectos en consultas literales (nDCG 1.0): «Aedes aegypti» devuelve solo el documento del dengue. BM25 supera por poco a TF-IDF (0.868 vs 0.860) por la saturación de frecuencia y la normalización por longitud, que con documentos de longitud casi uniforme aportan poco. Su debilidad es el vocabulario: «enfermedad transmitida por zancudos» no comparte palabras con el texto del dengue (que no dice «zancudo»); el acierto del primer lugar se debe a que «enfermedad» sí aparece, y el resto del ranking es ruido léxico (roya del café, fiesta).

Embeddings son los mejores en consultas semánticas (0.856), pero los peores en léxicas (0.649): promedian vectores y diluyen los nombres propios y términos raros. «Aedes aegypti» sí trae el documento correcto primero, pero rellena con documentos sin relación (jaguar, inundación); «cacao y chocolate» pone un documento de tamales antes que uno del cacao. Dan resultados siempre, aun cuando no hay nada relevante, y no distinguen bien lo que es una coincidencia exacta.

Híbrido RRF obtiene el mejor equilibrio (nDCG 0.908, MRR 1.000, R@5 0.790): mantiene el 1.0 en léxicas y sube las semánticas a 0.900, porque el documento que ambas listas coinciden en rankear alto gana sobre el que solo una propone. En consultas mixtas (0.826) no mejora a BM25 (0.830): ahí el aporte de los embeddings es neutro. «Temblor fuerte» ilustra la fusión: BM25 pone el simulacro primero, los embeddings el sismo, y RRF coloca ambos relevantes en el top 3.

4. Latencia p50/p95 y 50 usuarios concurrentes

Método. Generador de carga asíncrono (httpx) con N usuarios en bucle cerrado: cada uno elige consulta y motor al azar entre las 20 consultas y los 4 motores, espera la respuesta, hace una pausa aleatoria de 250–750 ms (tiempo de lectura de una persona) y repite durante 15 s. Se probaron 1, 10, 25 y 50 usuarios, más una variante de estrés sin pausa. Se reporta la latencia de extremo a extremo (cliente) y la del servidor (solo la búsqueda). **Entorno:** 2 vCPU, 8 GB de RAM, un solo proceso de uvicorn; servidor y generador de carga comparten la misma máquina, por lo que compiten por CPU y los números del estrés son pesimistas.

Usuarios	Peticiones	Errores	req/s	p50 (ms)	p95 (ms)	p99 (ms)	p50 / p95 búsqueda en servidor (ms)
1	29	0	1.9	3.84	5.20	7.11	0.30 / 0.63
10	302	0	19.4	4.00	6.88	22.03	0.30 / 0.42
25	761	0	48.8	4.45	8.50	52.95	0.31 / 0.43
50	1513	0	97.0	4.41	12.20	67.57	0.27 / 0.39

Tabla 1. Carga con pausa de ~500 ms (comportamiento de usuarios reales).

Usuarios	Peticiones	Errores	req/s	p50 (ms)	p95 (ms)	p99 (ms)
1	3470	0	347.0	2.74	4.06	5.15
10	5595	0	558.5	15.29	36.69	58.33
25	4299	0	428.5	35.92	181.05	273.77
50	3512	0	348.0	96.82	442.20	738.95

Tabla 2. Carga sin pausa (cada usuario dispara la siguiente consulta al recibir la respuesta).

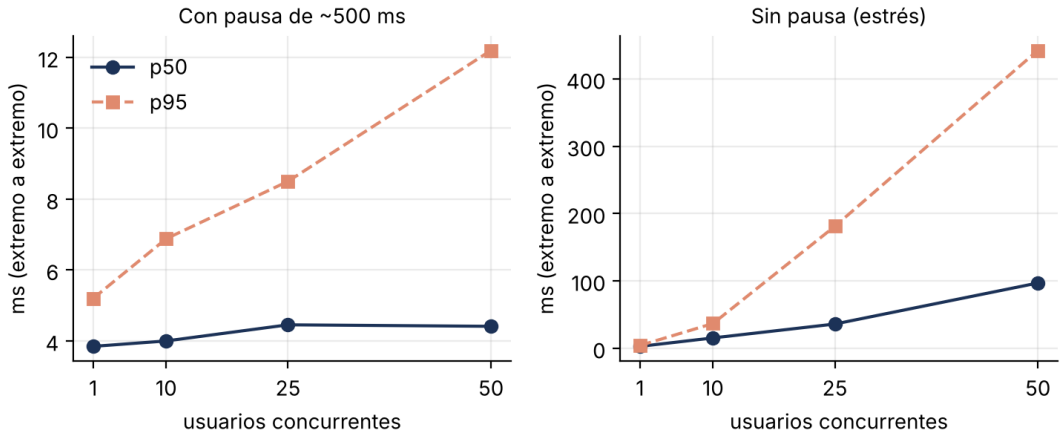


Figura 2. Latencia de extremo a extremo vs. usuarios concurrentes.

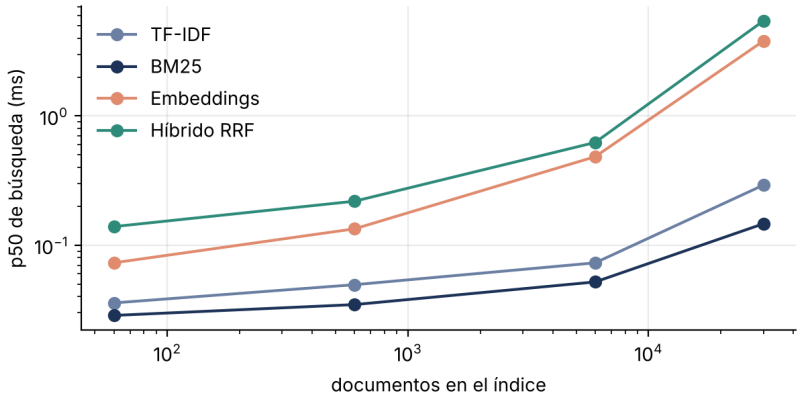
Lectura. Con 50 usuarios y pausa realista el sistema respondió **sin errores** con p50 de 4.41 ms y p95 de 12.20 ms (≈97 req/s); la búsqueda en sí toma 0.1–0.5 ms en el servidor, así que casi todo el tiempo es red local, FastAPI y serialización. Por motor, con 50 usuarios (extremo a extremo, p50/p95): TF-IDF 4.28/13.59 ms; BM25 4.24/11.52 ms; Embeddings 4.63/9.63 ms; Híbrido RRF 4.51/11.70 ms. Las diferencias entre motores (≈0.3 ms) son menores que el ruido de la red, por lo que **a esta escala la latencia no debe ser el criterio para escoger motor**. En estrés el sistema se satura en ≈350–560 req/s: la cola de peticiones, no el cálculo, hace crecer p95 (442 ms con 50 usuarios), un régimen mucho más duro que el uso humano. El modo threadpool vs. inline de FastAPI no mostró una diferencia concluyente en este tamaño de corpus.

5. Escalabilidad del corpus

Como 60 documentos no distinguen los motores por costo, se replicó el corpus a 600, 6,000 y 30,000 documentos y se midió la búsqueda en proceso (150 consultas por motor).

Documentos	Indexado (s)	TF-IDF p50/p95	BM25 p50/p95	Embeddings p50/p95	Híbrido p50/p95
60	0.12	0.04 / 0.07	0.03 / 0.05	0.07 / 0.14	0.14 / 0.21
600	0.58	0.05 / 0.11	0.03 / 0.08	0.13 / 0.23	0.22 / 0.35
6000	5.71	0.07 / 0.14	0.05 / 0.09	0.48 / 0.59	0.62 / 0.76
30000	29.9	0.29 / 0.49	0.15 / 0.24	3.80 / 6.52	5.44 / 6.99

Tabla 3. Latencia de búsqueda en ms según el tamaño del corpus.



Los motores léxicos crecen poco (el índice invertido solo toca los documentos que contienen los términos: 0.15 ms con 30,000), mientras que los embeddings y el híbrido crecen casi linealmente (3.8 y 5.4 ms) porque multiplican la consulta por toda la matriz N×300. Con corpus mucho mayores los embeddings requerirían un índice aproximado (ANN, p. ej. HNSW). Nota: esta réplica repite los textos, por lo que mide costo y no calidad.

6. Criterio de decisión

Se decidió con tres criterios, en este orden: (1) **calidad** (nDCG@5/MRR global y por tipo de consulta), (2) **robustez** (qué pasa cuando falla un componente o la consulta es rara) y (3) **costo** (latencia, memoria y complejidad de despliegue). Aplicados a los datos:

Criterio	TF-IDF	BM25	Embeddings	Híbrido RRF
Calidad global (nDCG@5)	0.860	0.868	0.762	**0.908**
Términos literales / nombres propios	Excelente	Excelente	Débil	Excelente
Paráfrasis / sinónimos	Débil	Débil	Buena	Buena
Latencia con 30,000 docs (p50)	0.29 ms	**0.15 ms**	3.8 ms	5.4 ms
Memoria / arranque	Mínima	Mínima	≈1 GB y ≈4 s (modelo)	≈1 GB y ≈4 s
Robustez	Silencio si no hay palabras comunes	Igual	Siempre responde, aunque sea ruido	Un componente mudo no lo rompe

Decisión: el híbrido RRF es la opción predeterminada del sistema, porque su ganancia en calidad (+0.04 sobre BM25, +0.15 sobre embeddings solos) cuesta milisegundos que ningún usuario percibe a esta

escala. **BM25** es la alternativa cuando se necesita ligereza (sin modelo de vectores, 1 GB menos de memoria) o los usuarios buscan sobre todo nombres, siglas y códigos. TF-IDF queda como línea base: no ofrece ventaja sobre BM25 salvo simplicidad. Los embeddings solos no se recomiendan como único motor.

7. Cuándo usar cada corpus

Corpus sintético de este proyecto (60 documentos). Sirve para desarrollar, probar y comparar motores de forma reproducible y controlada (sabemos exactamente qué documento es relevante); no para afirmar calidad en producción. Se usaría en pruebas automáticas, demostraciones y en la validación de que el sistema cumple el flujo completo.

Corpus real o scrapeado (p. ej. Wikipedia o noticias reales). Se usaría para evaluar con vocabulario, ruido y longitud reales, y para decidir de verdad el motor: el ruido (HTML, siglas, errores ortográficos) afecta distinto a cada motor. El scraper está en el repositorio (`scripts/scrape_corpus.py`) y la prueba de escalabilidad (30,000 documentos) indica que, con corpus grandes, convendría el híbrido con índice aproximado para la parte vectorial.

Corpus de las tareas 1–3 (14 documentos). Demasiado pequeño para medir latencia o estabilidad de las métricas; sirvió para entender las fórmulas, no para decidir.

8. Ventajas y desventajas observadas

Motor	Ventajas	Desventajas
TF-IDF	Simple, explicable (cada peso se puede calcular a mano); muy rápido; sin modelos.	No satura la frecuencia ni normaliza bien la longitud; ciego a sinónimos; sin resultados si no hay palabras comunes.
BM25	Mejor motor léxico; el más rápido y el que mejor escala (0.15 ms con 30,000 docs); precisión perfecta en nombres propios.	Mismos límites de vocabulario: «temblor» no encuentra «sismo»; depende del preprocesamiento (stemming, stopwords).
Embeddings	Capta paráfrasis y relaciones temáticas; mejor en consultas semánticas.	Diluye nombres propios y términos raros; siempre devuelve algo (ruido); ≈1 GB de modelo; crece linealmente con el corpus; vector nulo para siglas fuera de vocabulario.
Híbrido RRF	Mejor calidad global; combina las fortalezas sin ajustar pesos ni normalizar scores; tolera que un componente no responda.	El costo es la suma de ambos; las mejoras son pequeñas en consultas mixtas; es más difícil de explicar y depurar.

9. Alcance, limitaciones y web scraping

Web scraping (puntos extra): no se reclaman. Se escribió un scraper (respeta robots.txt, User-Agent identificable, 1 s de pausa, extracción con BeautifulSoup, ≥100 palabras, genera la lista de fuentes con URL, fecha y licencia CC BY-SA 4.0) con 80 artículos semilla de Wikipedia en español, pero el entorno donde se desarrolló el proyecto no tuvo acceso a es.wikipedia.org, por lo que solo se probó con HTML de muestra y nunca se ejecutó contra el sitio real. El corpus entregado es sintético y no hay fuentes externas que citar. Para obtener los puntos extra se ejecuta `make scrape` con red y se adjunta `data/sources_scraped.md`.

Otras limitaciones. (a) Los qrels derivan de reglas, no de anotadores independientes, y con 20 consultas las diferencias pequeñas (p. ej. TF-IDF vs. BM25) no son estadísticamente concluyentes. (b) Las pruebas de carga corren con servidor y cliente en la misma máquina de 2 vCPU. (c) El backend SBERT está implementado pero no se probó. (d) Los números de latencia absolutos dependen del hardware; son útiles por comparación entre motores y niveles de carga.

10. Reproducción y repositorio

`make install && make model && make test` (pruebas) · `make run` (servidor en el puerto 8000 con el frontend en «/») · `make eval` · `make loadtest` / `make loadtest-stress` · `make scalability` · `make report`. Repositorio: **CORZO_PEREZ_FRANCISCO_233296_C2_T4_INTEGRACION** (código, datos, pruebas, benchmarks y resultados crudos en `benchmark/results/`).

Fuentes. Corpus: textos originales redactados para el proyecto (sin fuentes externas). Vectores: modelo es_core_news_lg de spaCy (fastText, Explosion). Repositorio de la materia: github.com/RamsesCamas/DataMining_Slices_and_Code.